DeFuzz: Invariant-Guided Agentic Testing of Compiler-Instrumented Defenses

Abstract— Compiler-instrumented defenses (e.g., stack canaries, Control-Flow Integrity (CFI), and `_FORTIFY_SOURCE`) protect software against security threats, such as memory corruption, control-flow hijacking, and data leakage. However, these defenses can fail silently: the compiler produces a working binary without any warnings or crashes, yet the security protection is quietly weakened or broken. For example, in CVE-2023-4039, dynamic stack allocation allowed an overflow to bypass the GCC stack canary on AArch64. Existing compiler testing tools fail to detect such bugs. Conventional crash and differential testing only check whether a program produces correct outputs, leaving a fundamental oracle gap when a defense is silently weakened or broken.

In this paper, we present DEFUZZ, an invariant-directed agentic framework for discovering silent failures in compiler-instrumented defenses. DEFUZZ first extracts security invariants from compiler source code, Instruction Set Architecture (ISA) specifications, and historical bug fixes using a hybrid CoT and RAG pipeline. It then maps these invariants to compiler passes and decision branches, locating where defenses are implemented and controlled. To detect violations reliably, DEFUZZ formalizes these invariants in a predicate IR and lowers them into programmatic bug oracles, implementing them as static and dynamic checkers across target ISAs. All bug verdicts are decided strictly by these checkers, with zero LLM involvement in bug adjudication. Guided by the mapped compiler locations, specialized LLM agents perform directed fuzzing to hit targeted defense branches, synthesizing and mutating seeds to trigger invariant violations under oracle evaluation. We evaluated DEFUZZ on GCC and LLVM across multiple ISAs. DEFUZZ discovered 126 previously unknown silent failures in compiler-instrumented defenses, of which 34 have been confirmed upstream, including 11 fixed. Some of these bugs were rated as high-severity vulnerabilities. These results show that DEFUZZ effectively uncovers critical silent defense bugs that evade existing compiler testing techniques.

I. INTRODUCTION

Compilers deploy a wide range of compiler-instrumented defenses to harden software against security threats, such as memory corruption, control-flow hijacking, and sensitive data leakage. During compilation, these defenses insert runtime bounds checks, enforce stack layout rules, or clear registers to block exploit primitives. However, implementing these defenses accurately inside production compilers like GCC and LLVM is difficult. Defense code must interact with complex optimization passes and translate abstract protection rules into machine instructions across diverse target ISAs. This translation relies on strict assumptions about object bounds, instruction semantics, register conventions, and stack frame layouts. When the implementation of a defense mechanism fails to match these ISA or optimization assumptions, its security protection breaks from within.

When a defense implementation goes wrong, it often causes **silent defense failures**: the compiler produces a working binary without any warnings or crashes, yet the security protection is quietly weakened or broken. Because the compiled program still runs normally on benign inputs, standard functional tests pass completely, leaving the software exposed to attack. For example, in **CVE-2023-4039** [1], GCC on AArch64 placed the saved frame pointer and return address at the bottom of the fixed stack frame, below the stack canary. When a function used `alloca` or a variable-length array, GCC allocated that dynamic buffer at a lower stack address right below the saved return address. An upward buffer overflow from the dynamic region could then overwrite the saved return address without ever touching the canary.

Existing compiler testing approaches fail to catch such silent bugs due to two fundamental barriers in prior work. The first barrier of existing fuzzers is the **oracle gap**. Widely used compiler fuzzers (e.g., Csmith [2] and YARPGen [3]) check for compiler crashes or output mismatches across optimization levels. Because a binary with a broken defense still produces correct outputs on benign inputs, crash and differential oracles remain blind when a defense is weakened or broken. The second barrier in prior tools is locating defense-related compiler branches and satisfying their triggering conditions. Traditional directed fuzzers can reach given target branches, but they cannot tell which branches inside millions of lines of compiler code control a defense. Even after reaching a defense branch, exposing a silent failure requires satisfying defense-specific semantic conditions, such as opaque object bounds or cross-pass length propagation, rather than generic control-flow or data-flow coverage. For instance, in a GCC bug we uncovered (PR 125079 [4]), triggering silent `_FORTIFY_SOURCE` relaxation in the `strlen` pass requires pairing `__strcat_chk(dst, src, objsize)` on an unknown-length buffer with a subsequent `strlen(dst)` call that triggers the rewrite into `__stpcpy_chk`. Random mutators rarely synthesize such coupled defense semantics. Meanwhile, building static analyzers demands heavy manual effort from experts to craft custom rules for each defense mechanism and target ISA, while unconstrained LLM agents suffer from severe hallucinations and false bug reports.

These barriers raise a central question: how can we systematically detect silent defense failures when neither the buggy compiler nor direct LLM judgment provides a reliable oracle? To answer this question, we present **DEFUZZ**, an invariant-directed agentic framework for testing compiler-

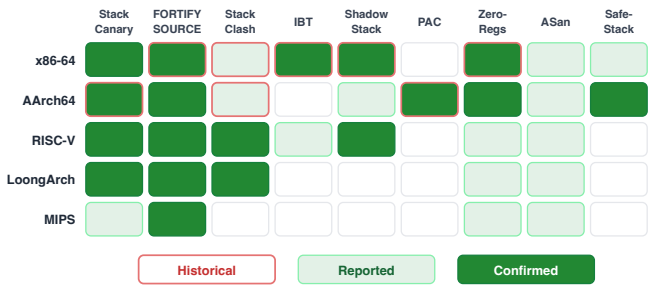

Fig. 1. Partial matrix of silent compiler defense bugs across representative defense mechanisms and target ISAs.

instrumented defenses. Our core insight is that although defense implementations vary across backends, their intended security guarantees follow target-independent *security invariants*. DEFUZZ translates these invariants into programmatic bug oracles implemented as ISA-specific static and dynamic checkers, closing the oracle gap. Because LLM-assisted translation can introduce errors, DEFUZZ formalizes each invariant in a predicate IR with explicit applicability and safety semantics, deterministically lowering validated specifications into executable checkers. To trigger defense-critical logic, DEFUZZ maps each invariant to its corresponding defense-related compiler branches and guides LLM agents to synthesize inputs that satisfy both branch conditions and invariant preconditions. All bug verdicts remain strictly with deterministic checkers, avoiding both the blind spots of random mutators and LLM adjudication hallucinations.

Building DEFUZZ requires solving three main challenges. The first challenge is establishing reliable security invariants: because the compiler itself has bugs, we cannot trust it. Relying on a compiler's own behavior or implementation code as the ground truth creates circular self-validation. Our key idea is to separate intended security contracts from buggy compiler implementations by anchoring invariants in external ISA manuals, documented design assertions, and confirmed security patches, while abstracting past fixes into transferable failure patterns across defenses. The second challenge is checking these invariants with high accuracy across diverse ISAs, where distinct registers and stack layouts cause direct LLM judgment to hallucinate. DEFUZZ addresses this by formalizing each invariant in a predicate IR parameterized by target ISA specifications, lowering it into ISA-specific static and dynamic checkers. Dual static-dynamic cross-checking and differential baseline builds further verify each target, ensuring reliable verdicts with **zero LLM involvement in bug adjudication**. The final challenge is steering test generation to both locate defense branches and satisfy their semantic preconditions without being stripped by early optimizations. DEFUZZ maps each invariant to its controlling compiler passes and branches via call-graph and AST analysis, inserting compile-time probes to trace branch execution. Guided by branch traces and invariant preconditions, LLM agents iteratively synthesize and mutate seeds to expose deep defense flaws.

We evaluated DEFUZZ on GCC and LLVM across x86-64, AArch64, and RISC-V, spanning diverse defenses such as stack canaries, CFI, shadow stacks, register scrubbing, and `_FORTIFY_SOURCE`. DEFUZZ extracted 915 validated security invariants and transferred historical failure patterns across defense mechanisms and backends. During fuzzing, branch-guided seed synthesis hit substantially more target defense branches than unguided baselines, while our formally specified checkers identified violations with high accuracy and avoided the false reports produced by unconstrained LLM agents. Overall, DEFUZZ discovered **126 previously unknown silent failures in compiler-instrumented defenses**, of which **34 have already been confirmed upstream, with 11 fixed** (Figure 1), including bugs rated as high-severity vulnerabilities. These results show that DEFUZZ effectively uncovers critical silent defense bugs that evade existing compiler testing techniques.

In answering this central question, this paper makes the following contributions:

- **Automated Invariant Extraction:** We decouple security contracts from buggy compiler outputs by extracting security invariants from ISA specifications, design assertions, and historical bug fixes, transferring failure patterns across defenses.
- **Validated Translation into Multi-ISA Programmatic Oracles:** We formalize security invariants in a predicate IR and lower them into formally specified static and dynamic checkers across target ISAs, verifying compiled code with high accuracy and zero LLM involvement in bug adjudication.
- **Branch-Guided Agentic Fuzzing:** We map invariants to defense-related compiler passes and branches, and guide specialized LLM agents with compile-time branch traces to synthesize seeds that satisfy defense triggering conditions.
- **New Silent Defense Bug Discovery:** We evaluate DEFUZZ on GCC and LLVM across multiple ISAs and discover 126 previously unknown silent failures in compiler-instrumented defenses, with 34 confirmed upstream, including 11 fixed.